CPRE 4870/5870 - Lab 02

Cameron Gilbertson, Tyler Bibus

3.2 Layer Operations

3.2.1: For the neural network model used in Lab 1, list the different types of layers present and write the mathematical formulation of each layer type. When writing the mathematical notation, use the tensor index notation introduced in the textbook (see equations 2.2 and 2.3).

1. Convolutional Layer

$$O_{nmpq} = (\sum_{crs} I_{nc(U_{p+r})(U_{q+s})} F_{mcrs}) + b_m$$

2. Pooling Layer

$$O_{nmwh} = max_{m=0,\ldots,M-1} max_{h=0,\ldots,H-1} max_{w=0,\ldots,W-1} I_{(N,C,stride * h + m,\, stride * w +}$$

3. Flattening Layer

$$O_i = (\sum_{chw} I_{nchw})$$

4. Dense Layer

$$O_m = (\sum_c I_c F_c) + b_m$$

5. SoftMax Layer

$$O_n = exp(I_n) / \sum_n exp(I_n)$$

3.2.2: Also include a brief description of each variable, its use/purpose, and how that is represented in our model. Provide the specific values of the relevant (to a particular layer) parameters listed in Table 2.1 of the textbook.

Shape Parameter	**Description**
N	Batch Size
M	Number of output channels
C	Number of input channels
H/W	Size of input height/width
R/S	Filter size height/width
P/Q	Output size height/width

3.2.3: Report the result of appropriate comparison functions and justify that the value is acceptable.

```
[Info]: --- Running Layer Test 0---
Opened binary file data/image_0.bin
Timer Layer Inference: elapsed=7.707000ms
Opened binary file data/image_0_data/layer_0_output.bin
Comparing Outputs (Cosine Similarity): True 100%  (1)
[Info]: --- Running Layer Test 2---
Opened binary file data/image_0_data/layer_1_output.bin
Timer Layer Inference: elapsed=0.123000ms
Opened binary file data/image_0_data/layer_2_output.bin
Comparing Outputs (Cosine Similarity): True 100%  (1)
[Info]: --- Running Layer Test 9---
Opened binary file data/image_0_data/layer_8_output.bin
Timer Layer Inference: elapsed=0.001000ms
Opened binary file data/image_0_data/layer_9_output.bin
Comparing Outputs (Cosine Similarity): True 100%  (1)
[Info]: --- Running Layer Test 10---
Opened binary file data/image_0_data/layer_9_output.bin
Timer Layer Inference: elapsed=1.057000ms
Opened binary file data/image_0_data/layer_10_output.bin
Comparing Outputs (Cosine Similarity): True 100%  (1)
[Info]: --- Running Layer Test 11---
Opened binary file data/image_0_data/layer_10_output.bin
Timer Layer Inference: elapsed=0.040000ms
Opened binary file data/image_0_data/layer_11_output.bin
Comparing Outputs (Cosine Similarity): True 100%  (1)
```

Layer 0 (Convolutional Layer): 100%
Layer 2 (MaxPooling Layer): 100%
Layer 9 (Flattening Layer): 100%
Layer 10 (Dense Layer + ReLU): 100%
Layer 11 (Dense Layer + Softmax): 100%

3.3 Analysis

3.3.1: Estimate the size of the memory needed for each layer and how many MAC computations are required for each layer.

For a convolutional Layer the formula is roughly:
Memory: weights + biases + inputs + outputs

Weights: K x K x Cin x Cout x 4bytes (f32)
Biases: Cout x 4bytes
Inputs: Cin x H x W x 4bytes
Outputs: Cout x Hout x Wout x 4 bytes

For the total MACs the equations is roughly
M x Hout X Wout x C x R x S

Conv Layer 1:

Weights: 5x5x3x4 = 300 Bytes
Biases: 32 x 4 = 128 bytes
Inputs: 64 x 64 x 3 x 4 = 49152 bytes
Output: 60 x 60 x 32 x 4 = 460800 bytes

Total Memory = 510,380 bytes
Memory for parameters is just the weights + bias is 428 Bytes.
MAC for this layer would be: 32 x 60 x 60 x 3 x 5 x 5 = 8640000 MACs

Conv Layer 2:

Weights: 5x5x32x4 = 3200 Bytes
Biases: 32 x 4 = 128 bytes
Inputs: 60 x 60 x 32 x 4 = 460,800 bytes
Output: 56 x 56 x 32 x 4 = 401,408 bytes

Total Memory = 865,536 bytes
Memory for parameters is just the weights + bias is 3328 Bytes.
MAC for this layer would be: 32 x 56 x 56 x 32 x 5 x 5 = 80281600 MACs

MPL 1:

Pool Size = 4 bytes (1 variable)
Stride = 4 bytes
Inputs = 56 x 56 x 32 x 4 = 401408 bytes
Outputs = 28 x 28 x 32 x 4 = 100352 Bytes

Total Memory = 501,768 Bytes
No MACs, but does take fMAX()

Conv 3:

Weights: 3x3x32x4 = 1152 Bytes

Biases: 64 x 4 = 256 bytes

Inputs: 28x28x32x4 = 100352 Bytes

Output: 26x26x64x4 = 173056 Bytes

Total Memory = 274816 bytes

Memory for parameters is just the weights + bias is 1,408 Bytes.

MAC for this layer would be: 64 x 26 x 26 x 32 x 3 x 3 = 12460032 MACs

Conv 4:

Weights: 3x3x64x4 = 2304 Bytes

Biases: 64 x 4 = 256 bytes

Inputs: 28x28x32x4 = 100352 Bytes

Output: 26x26x64x4 = 173056 Bytes

Total Memory = 275,968 bytes

Memory for parameters is just the weights + bias is 2560 Bytes.

MAC for this layer would be: 25x26x64x3x3x64 = 23961600 MACs

MPL 2:

Pool Size = 4 bytes (1 variable)

Stride = 4 bytes

Inputs = 24x24x64x4 = 147456 Bytes

Outputs = 12x12x64x4 = 36864 Bytes

Total Memory = 184,328 Bytes

No MACs, but does take fMAX()

Conv 5:

Weights: 3x3x64x4 = 2304 Bytes

Biases: 64 x 4 = 256 bytes

Inputs: 12x12x64x4 = 36864 Bytes

Output: 10x10x64x4 = 25600 Bytes

Total Memory = 65,024 bytes
Memory for parameters is just the weights + bias is 2560 Bytes.
MAC for this layer would be: 10x10x64x3x3x64 = 3686400 MACs

Conv 6:

Weights: 3x3x64x4 = 2304 Bytes
Biases: 128 x 4 = 512 bytes
Inputs: 10x10x64x4 = 25600 Bytes
Output: 8x8x128x4 = 32768 Bytes

Total Memory = 61184 bytes
Memory for parameters is just the weights + bias is 2816 Bytes.
MAC for this layer would be: 8x8x128x3x3x64 = 4718592 MACs

MPL 3:

Pool Size = 4 bytes (1 variable)
Stride = 4 bytes
Inputs = 8x8x128x4 = 32768 Bytes
Outputs = 4x4x128x4 = 8192 Bytes

Total Memory = 40968 Bytes
No MACs, but does take fMAX()

dense 1:
Memory:
K x F32 = 256 x 4 = 1024 Bytes
Bias = 200 x 4 = 800 Bytes
Input = 2048x4 = 8192 Bytes
Output = 256x4 = 1024 Bytes
Total Memory = 11,040 Bytes
OutputSize x InputSize = 2048 x 256 = 524288 MACs

dense 2:

Memory:
K x F32 = 256 x 4 = 1024 Bytes
Bias = 200 x 4 = 800 Bytes
Input = 256x4 = 1024 Bytes
Output = 200x4 = 800 Bytes
Total Memory = 3,648 Bytes
OutputSize x InputSize = 200 x 256 = 51200 MACs

3.3.2: Verify that your outputs match the expected outputs sourced from the Tensorflow implementation

```
[Info]: --- Running Inference Test ---
Opened binary file data/image_0.bin
Timer Full Inference: elapsed=88.583000ms
Opened binary file data/image_0_data/layer_11_output.bin
Comparing Outputs (Cosine Similarity): True 100%  (1)
```

3.3.3: What was the final maximum error you saw/the error tolerance you used in your verification process for comparing your output?

```
[Info]: --- Running Basic Test ---
Opened binary file ./data/image_0.bin
Comparing image 0 to itself (max error): 1
Comparing image 0 to itself (T/F within epsilon 0.001): false

Change a value by 0.1 and compare again
Comparing Outputs (Cosine Similarity): True 99.9993%  (0.999993)
Change a value by 0.1 and compare again...
Comparing Outputs (Cosine Similarity): True 99.9981%  (0.999981)
[Info]: --- Running Layer Test 0---
```

Our maximum error ended up being 0.0019% for the basic test. Our Inference test and layer tests came back with 100% accuracy.

3.3.4: How did you come to that value and what is a reasonable error range?

This values was provided from running the toolflow. For the layers and Inference it should match 100% with no range. The toolflow gets the output by getting the real output (binary file) and comparing it to the generated output from the user code.

3.3.5: Where does this error come from and should there be any at all?

The error from the basic test is intentional as it modifies an image pixel by 0.1 two times (hence why the comparison has error, and why it is so small. For the layers and inference there should be no error, as the architecture of the model should be the same. Any difference would indicate incorrect calculations.

3.4 Profiling C++ Implementation

3.4.1: Execute your inference application and report the time taken for a single inference on both your x86 lab machine and the ZedBoard. Compare these to the time taken when a single inference is executed using the Tensorflow backend. You could use a Tensorboard to measure the performance. If you see a difference in execution time, why do you think there is a difference?

Lab Computers

```
[Info]: --- Running Inference Test ---
Opened binary file data/image_0.bin
Timer Full Inference: elapsed=88.583000ms
Opened binary file data/image_0_data/layer_11_output.bin
Comparing Outputs (Cosine Similarity): True 100%  (1)
```

ZedBoard

```
[Info]: --- Running Inference Test ---
xsdb% Opened binary file data/image_0.bin
xsdb% Timer Full Inference: elapsed=3013.000000ms
xsdb% Opened binary file data/image_0_data/layer_11_output.bin
xsdb% Comparing Outputs (Cosine Similarity): True 100%  (1)
```

Here we noticed a significant difference between running inference on the Lab Computers compared to the ZedBoard. The Lab Computers had a total time of 88.5 milliseconds compared to the ZedBoard which had a total time of 3013.0 milliseconds. We believe that the difference in execution time comes down to processing power of the Lab Computers compared to the ZedBoard. The Lab Computers have much more processing power than the ZedBoard leading to decrease in execution time.

3.4.2: Figure out which layers take most of the computation time. Plot the layerwise execution time fraction and report it. State possible reasons why a particular layer takes significant computation time? Does this match your Lab 1 results from Tensorboard? Which layers may be ripe for optimization? Why?

```
Samples: 7M of event 'cpu_core/cycles/Pu', Event count (approx.): 794048700
Overhead  Command  Shared Object        Symbol
  91.94%  ml       ml                   [.] ML::ConvolutionalLayer::computeNaive(ML::LayerData const&) const
   2.02%  ml       ml                   [.] ML::DenseLayer::computeNaive(ML::LayerData const&) const
   1.35%  ml       ml                   [.] float ML::LayerData::compare<float>(ML::LayerData const&) const
   0.66%  ml       ml                   [.] ML::MaxPoolingLayer::computeNaive(ML::LayerData const&) const
```

From our perf data, it shows that the convolutional layer takes much longer than the other layers. While there are more convolutional layers this is not a proportional amount of performance. In Lab 1, the convolutional layer was also the largest cost in terms of performance. This matches our results from the perf data. Overall the layer that would receive the most benefit from optimizations would be the convolutional layer because of the high performance impact and with the frequency of the convolutional layers.